

CET251: AI Course Practical Project Brief

Course: CET251 - Introduction to Artificial Intelligence

Program: B.Sc. Computer Science

Level: Second-Year Students

Institution: El Sewedy University of Technology

Project Duration: 4 Weeks

Team Size: Up to 5 students per team

Purpose

The practical project is designed to help students apply core AI concepts from the course to realistic, manageable problems through a working prototype. Each team will select one approved project and implement it using Python and lightweight open-source tools.

Each project must:

- solve a clearly defined real-world problem,
- use at least **two AI concepts** from the course,
- include a **small neural-network or starter deep-learning component**,
- and produce a working demo with evaluation.

General implementation rules

- Use **Python** as the main language.
- Keep the project **small, complete, and testable**.
- Use **synthetic data, open data, or a simple simulation**.
- Do not build a large production app; focus on **AI logic and a working prototype**.
- A simple interface is enough: command line, notebook, Streamlit, or basic Tkinter/web page.
- Every team must compare at least **two approaches** where applicable, such as BFS vs A*, minimax vs alpha-beta, or manual rules vs learned model.

Approved Project List with Detailed Instructions

1) Smart Parking Finder & Route Advisor

Tagline: Find the best parking spot and the fastest route to your destination.

Scenario

Students, staff, and visitors often spend time searching for parking during busy hours. The closest parking area may already be full, reserved, or far from the actual destination once walking distance

is considered. This project builds an assistant that recommends the best parking area and then guides the user to the destination.

What the system should do

The user enters:

- destination building,
- arrival time,
- whether they prefer nearest parking, fastest walking route, or highest chance of finding a space.

The system should:

- estimate which parking areas are likely available,
- choose a good parking lot,
- compute the route from that lot to the destination,
- and show an explanation for the recommendation.

Core AI concepts

- Agents & Environments
- BFS / DFS
- A* Search

Neural-network add-on

Train a small model to predict parking occupancy using simple features such as:

- hour of day,
- weekday,
- parking lot ID,
- nearby class density,
- event/no event indicator.

Inputs

- campus parking lots,
- parking capacities,
- road/walking graph,
- destination building,
- simulated occupancy data.

Outputs

- recommended parking lot,
- predicted availability,
- walking route,
- estimated walking time,
- alternative parking options.

Suggested tools

- Python
- NetworkX
- pandas
- matplotlib
- scikit-learn or Keras

Minimum scope

- 3 to 5 parking lots
- 10 to 20 campus nodes
- occupancy prediction on synthetic data
- BFS and A* comparison

Stretch ideas

- accessible parking mode,
- staff/student priority mode,
- live re-routing if a lot becomes full.

Important instruction

Do not spend time building a fancy map. A simple graph visualization is enough. Focus on search logic and decision quality.

2) Smart Study Room & Lab Booking Assistant

Tagline: Help students find the best room at the right time.

Scenario

Student teams need rooms for studying, project meetings, or lab work. Some rooms have equipment, some have capacity limits, and many time slots are already occupied. The project builds a booking assistant that recommends feasible room/time options.

What the system should do

The user enters:

- number of students,
- preferred day and time range,
- duration,
- required features such as PCs, projector, or lab equipment.

The system should:

- search for valid rooms,
- avoid double booking,

- enforce capacity and feature constraints,
- recommend alternatives if the preferred slot is not available.

Core AI concepts

- CSP
- Backtracking
- MRV / degree heuristics
- Agents & Environments

Neural-network add-on

Predict room demand or noisy hours using features such as:

- day of week,
- hour,
- room type,
- exam week / normal week,
- historical synthetic booking load.

Inputs

- room list,
- room capacities,
- room features,
- booking schedule,
- user request.

Outputs

- valid room-time assignments,
- ranked alternatives,
- explanation of rejected options.

Suggested tools

- Python
- pandas
- python-constraint or custom backtracking
- scikit-learn / Keras
- Streamlit optional

Minimum scope

- 6 to 10 rooms
- one-week schedule
- at least 3 constraints
- one demand-prediction model

Stretch ideas

- waiting list,
- recurring booking,
- group preference score.

Important instruction

Clearly define the CSP:

- variables,
 - domains,
 - constraints.
- This should appear in the report.

3) Student Course Registration Helper

Tagline: Build valid semester schedules and avoid overload.

Scenario

Students often choose courses without understanding prerequisite chains, timetable clashes, or workload balance. This project builds a smart helper that generates valid semester schedules and warns about risky combinations.

What the system should do

The user enters:

- courses already passed,
- available course sections,
- target credit load,
- preferences such as no early classes or no Friday classes.

The system should:

- check prerequisites,
- avoid timetable conflicts,
- generate feasible schedules,
- rank them using selected criteria.

Core AI concepts

- CSP
- Graph search over prerequisite structure
- Agents & Environments

Neural-network add-on

Predict overload or difficulty risk using features such as:

- number of labs,
- number of technical courses,
- total credits,
- previous GPA category,
- morning/evening distribution.

Inputs

- course catalog,
- prerequisite graph,
- section timetable,
- student history.

Outputs

- valid schedules,
- warnings for conflicts,
- risk score for each schedule,
- best-suggested option.

Suggested tools

- Python
- NetworkX
- pandas
- scikit-learn / Keras

Minimum scope

- 10 to 15 courses
- 2 to 3 semesters or one level
- at least 3 student test profiles

Stretch ideas

- graduation path planning,
- “what if I fail course X?” mode,
- balanced schedule recommendation.

Important instruction

Do not try to model the full university. Use a small realistic subset.

4) Campus Lost-and-Found Matcher

Tagline: Match lost-item reports with found-item reports intelligently.

Scenario

Students lose IDs, chargers, water bottles, notebooks, and headphones. Matching lost and found reports manually is time-consuming. This project builds a matching assistant.

What the system should do

The user can submit:

- lost item description,
- place,
- time,
- color,
- category.

The system should:

- compare lost and found reports,
- rank likely matches,
- explain why the system suggested them.

Core AI concepts

- Agents & Environments
- Informed search / ranked matching
- Optional CSP filtering

Neural-network add-on

A simple text model or classifier to identify item type from description.

Inputs

- lost item reports,
- found item reports,
- item categories,
- times and locations.

Outputs

- top matching candidates,
- confidence/ranking score,
- reasons for match.

Suggested tools

- Python
- pandas
- scikit-learn
- cosine similarity / TF-IDF

Minimum scope

- 50 to 100 sample reports
- at least 4 item categories
- ranked matching interface

Stretch ideas

- image upload of found item,
- Arabic/English descriptions,
- duplicate report detection.

Important instruction

Keep categories simple at first: ID card, charger, bottle, notebook, headphones, keys.

5) Mini Warehouse Picking Optimizer

Tagline: Plan the fastest route to collect items in a warehouse.

Scenario

A worker needs to collect items from different shelves. Some aisles may be blocked, and poor routing wastes time. This project builds a warehouse route planner.

What the system should do

Given:

- a warehouse layout,
- start location,
- list of required items,
- blocked aisles,

the system should:

- compute a route,
- compare alternative search strategies,
- estimate time to finish the order.

Core AI concepts

- Agents & Environments

- BFS
- A* Search

Neural-network add-on

Predict picking time based on:

- number of items,
- route length,
- congestion level,
- blocked aisle count.

Inputs

- warehouse grid or graph,
- shelf locations,
- items per shelf,
- picking order.

Outputs

- path sequence,
- estimated time,
- total travel cost,
- alternate route if aisle blocked.

Suggested tools

- Python
- NetworkX
- matplotlib
- scikit-learn

Minimum scope

- 8 to 12 shelf locations
- single picker
- single order with 3 to 6 items
- one blocked-path scenario

Stretch ideas

- multiple orders,
- multiple pickers,
- priority items.

Important instruction

A simple 2D grid is enough. Do not overcomplicate the warehouse model.

6) Energy Project: Smart Room AC Scheduler

Tagline: Reduce classroom energy waste while maintaining comfort.

Scenario

AC units may run in empty rooms or for longer than necessary. This project builds a scheduling assistant that recommends when classroom AC should be turned on or off.

What the system should do

Using:

- room timetable,
- room size,
- expected occupancy,
- operating rules,

the system should:

- recommend AC start and stop times,
- reduce unnecessary use,
- compare energy-saving scenarios.

Core AI concepts

- CSP
- Agents & Environments
- Optional informed search

Neural-network add-on

Predict room occupancy using:

- day,
- hour,
- room type,
- timetable density,
- previous usage pattern.

Inputs

- room schedules,
- room capacities,
- synthetic energy assumptions,
- occupancy data.

Outputs

- AC schedule,
- estimated energy use,
- energy-saving comparison,
- rooms with likely waste.

Suggested tools

- Python
- pandas
- matplotlib
- scikit-learn / Keras

Minimum scope

- 4 to 8 rooms
- one-week schedule
- simple energy formula
- before/after comparison

Stretch ideas

- peak-hour electricity pricing,
- comfort score,
- priority for lab rooms.

Important instruction

Use a simple transparent energy model and explain assumptions clearly.

7) Fitness Project: Smart Workout & Habit Planner

Tagline: Recommend a realistic weekly workout plan that fits student goals and schedule.

Scenario

Students often want to improve fitness, but their schedules, energy levels, and habits make it difficult to maintain a consistent plan. This project builds a workout and habit-planning assistant.

What the system should do

The user enters:

- fitness goal,
- available days and times,
- preferred workout style,
- beginner/intermediate level,

- optional constraints such as home workout only or short sessions.

The system should:

- generate a weekly plan,
- avoid unrealistic overload,
- balance workout types across the week,
- adapt the plan if one day is missed.

Core AI concepts

- CSP
- Agents & Environments
- Optional Genetic Algorithm for plan optimization

Neural-network add-on

Predict workout adherence likelihood or difficulty level using features such as:

- available time,
- sessions per week,
- workout intensity,
- previous completion history,
- goal type.

Inputs

- user goals,
- weekly availability,
- exercise list,
- workout constraints,
- optional history of completed sessions.

Outputs

- weekly workout plan,
- adherence or difficulty score,
- alternative plan if a session is missed,
- explanation of plan choices.

Suggested tools

- Python
- pandas
- python-constraint or custom solver
- scikit-learn / Keras
- Streamlit optional

Minimum scope

- one user profile at a time
- one-week plan
- at least 3 workout types
- one adherence or difficulty predictor

Stretch ideas

- meal or hydration reminder integration,
- progressive difficulty over multiple weeks,
- rest-day optimization.

Important instruction

Keep the project focused on **planning and recommendation**, not medical advice or diagnosis.

8) Game Project: Maze Solver Challenge

Tagline: Build a game agent that escapes faster and smarter.

Scenario

A game character must navigate a maze with walls, traps, and changing conditions. Students compare search algorithms in a game-like environment.

What the system should do

The agent should:

- start from a defined point,
- avoid obstacles and traps,
- reach the goal,
- and compare strategies using different algorithms.

Core AI concepts

- Agents & Environments
- BFS / DFS
- A* Search

Neural-network add-on

Predict risky nearby cells or trap probability from local features.

Inputs

- maze maps,
- start and goal positions,
- trap locations,
- optional moving obstacles.

Outputs

- path found,
- path length,
- runtime or node expansions,
- failure/success comparison.

Suggested tools

- Python
- pygame optional
- matplotlib or console grid
- scikit-learn / Keras

Minimum scope

- 3 different mazes
- 3 search algorithms
- 1 risk-prediction feature

Stretch ideas

- level generator,
- moving enemy,
- limited-visibility mode.

Important instruction

Game graphics are optional. Strong algorithm comparison matters more than appearance.

9) Game Project: Connect-4 AI Opponent

Tagline: Build a strategic AI that challenges human players.

Scenario

This project builds a playable Connect-4 game where the computer makes decisions intelligently using adversarial search.

What the system should do

The system should:

- allow human vs AI play,
- support multiple difficulty levels,
- evaluate board states,
- compare minimax and alpha-beta pruning.

Core AI concepts

- Adversarial Search
- Minimax
- Alpha-Beta Pruning
- Agents & Environments

Neural-network add-on

Train a small board-state evaluator using self-play or generated positions.

Inputs

- board state,
- legal moves,
- difficulty level.

Outputs

- next best move,
- win/loss/draw result,
- AI performance comparison.

Suggested tools

- Python
- numpy
- pygame or Tkinter optional
- scikit-learn / Keras

Minimum scope

- playable game
- minimax implementation
- alpha-beta implementation
- at least 2 difficulty levels

Stretch ideas

- explain AI move,
- AI vs AI tournament,
- learned heuristic evaluator.

Important instruction

Keep search depth manageable. Performance matters.

10) Emergency Evacuation Route Planner

Tagline: Guide people safely to exits during emergencies.

Scenario

In emergencies such as fire alarms or blocked corridors, people in different rooms need safe exit routes. This project builds an evacuation planner for a building.

What the system should do

Using:

- building map,
- exit locations,
- blocked areas,
- groups in rooms,

the system should:

- assign safe exit routes,
- avoid blocked paths,
- optionally distribute groups across exits.

Core AI concepts

- Agents & Environments
- BFS
- A* Search
- Optional CSP

Neural-network add-on

Predict corridor or exit congestion from occupancy features.

Inputs

- building graph,
- room occupancy,
- exit locations,
- blocked corridor scenarios.

Outputs

- evacuation paths,
- assigned exits,
- congestion estimate,

- comparison of plans.

Suggested tools

- Python
- NetworkX
- matplotlib
- scikit-learn

Minimum scope

- one building floor
- 8 to 15 rooms
- 2 exits
- 2 emergency scenarios

Stretch ideas

- multi-floor extension,
- wheelchair-accessible route,
- exit failure scenario.

Important instruction

Use a simple building model and focus on route logic and safety reasoning.

11) Smart Canteen Queue Manager

Tagline: Predict waiting times and recommend the best counter.

Scenario

At busy lunch hours, students do not know which queue is fastest. This project builds a smart queue assistant for a canteen or food court.

What the system should do

Given:

- current queue lengths,
- service speed at each counter,
- time of day,

the system should:

- estimate waiting time,
- recommend the best counter,
- compare against naive choice.

Core AI concepts

- Agents & Environments
- Rule-based decision making
- Optional CSP

Neural-network add-on

Predict waiting time from:

- queue length,
- hour,
- counter type,
- average service duration.

Inputs

- queue lengths,
- counter speeds,
- time,
- simple historical synthetic data.

Outputs

- predicted wait per counter,
- recommended counter,
- estimated service completion time.

Suggested tools

- Python
- pandas
- matplotlib
- scikit-learn

Minimum scope

- 3 counters
- 1 queue simulator
- wait-time prediction model

Stretch ideas

- special meal rush,
- prepaid vs cash counters,
- student preference mode.

Important instruction

Keep the simulation simple and measurable.

12) Delivery Time Slot Planner for a Small Shop

Tagline: Assign deliveries to drivers while reducing late orders.

Scenario

A small business has several customer orders, a few drivers, and limited delivery time windows. The project builds a planner that assigns orders and estimates late-delivery risk.

What the system should do

Given:

- orders,
- delivery zones,
- available drivers,
- time windows,

the system should:

- assign orders to drivers,
- propose a delivery sequence,
- detect risky or late deliveries.

Core AI concepts

- CSP
- Search
- Agents & Environments

Neural-network add-on

Predict delivery delay risk from:

- distance,
- order count,
- driver load,
- time of day.

Inputs

- driver list,
- order list,
- delivery zones,
- time constraints.

Outputs

- driver assignment,
- delivery sequence,
- risk flags,
- estimated lateness.

Suggested tools

- Python
- pandas
- matplotlib
- scikit-learn

Minimum scope

- 2 to 3 drivers
- 8 to 12 orders
- 3 delivery zones

Stretch ideas

- urgent orders,
- failed delivery rescheduling,
- traffic scenario.

Important instruction

Do not try to solve a full vehicle-routing problem. Keep it small and understandable.

13) Kaggle Project: Movie Recommendation & Viewing Planner

Tagline: Recommend what to watch next based on user taste and viewing patterns.

Scenario

Streaming platforms try to recommend movies that match user interests while still helping users discover new content. This project uses a small Kaggle movie-rating or movie-metadata dataset to build a recommendation prototype.

What the system should do

The user enters:

- a small list of watched or liked movies,
- preferred genres,
- optional preferences such as short duration, high rating, or family-friendly content.

The system should:

- recommend similar or personalized movies,
- rank recommendations,
- explain why the movies were suggested,
- and optionally build a short watch plan.

Core AI concepts

- Agents & Environments
- Informed search / ranked recommendation
- Optional Genetic Algorithm for optimizing recommendation lists or watch plans

Neural-network add-on

Predict whether a user is likely to like a movie using features such as:

- genre,
- runtime,
- average rating,
- release year,
- popularity,
- previously liked movies.

Inputs

- movie metadata dataset,
- user watched/rated list,
- genre tags,
- movie features.

Outputs

- ranked movie recommendations,
- predicted preference score,
- optional watch plan,
- explanation of recommendation logic.

Suggested tools

- Python
- pandas
- scikit-learn / Keras
- cosine similarity or nearest-neighbor methods
- Streamlit optional

Minimum scope

- one small Kaggle movie dataset

- at least 100 to 500 movies
- at least 2 recommendation methods
- one preference-prediction model

Stretch ideas

- cold-start user mode,
- genre diversity control,
- mood-based recommendations.

Important instruction

Choose a **small and clean Kaggle dataset**. Do not build a full industrial recommender system. Focus on a clear prototype and explain how recommendations are ranked.

14) Health Care Project: Clinic Appointment Scheduling Assistant

Tagline: Schedule appointments fairly while reducing waiting time.

Scenario

A small clinic needs to assign patients to appointment slots while considering doctor availability, visit type, and urgent cases. This project builds an intelligent scheduling assistant.

What the system should do

The user enters:

- patient requests,
- doctor availability,
- appointment type,
- urgency level,
- preferred day or time range.

The system should:

- assign patients to valid time slots,
- avoid clashes,
- prioritize urgent cases,
- recommend alternatives when a preferred slot is unavailable.

Core AI concepts

- CSP
- Backtracking
- Agents & Environments
- Optional Genetic Algorithm for improving schedule quality

Neural-network add-on

Predict no-show risk or expected appointment duration using features such as:

- patient category,
- appointment type,
- time of day,
- weekday,
- urgency level.

Inputs

- patient requests,
- doctor schedules,
- clinic rooms or resources,
- appointment durations,
- urgency tags.

Outputs

- appointment schedule,
- conflict-free assignments,
- priority handling decisions,
- alternative slot suggestions.

Suggested tools

- Python
- pandas
- python-constraint or custom backtracking
- scikit-learn / Keras
- Streamlit optional

Minimum scope

- 2 to 3 doctors
- one-day or one-week schedule
- at least 10 to 20 patient requests
- one prediction model for no-show or duration

Stretch ideas

- walk-in patient handling,
- specialist room constraints,
- fairness metrics for doctor load.

Important instruction

Keep the project focused on **operations and scheduling only**. Do not build a medical diagnosis system.

15) Collaboration Project: Team Project Formation & Task Planner

Tagline: Build balanced teams and assign project tasks intelligently.

Scenario

In project-based learning or workplace collaboration, teams need to be formed fairly while balancing skills, availability, and task preferences. This project builds a team-formation and task-allocation assistant.

What the system should do

The user enters:

- list of students or team members,
- skill levels,
- preferred roles,
- availability,
- project tasks.

The system should:

- form balanced teams,
- assign suitable tasks or roles,
- avoid major conflicts,
- and suggest alternative team structures if needed.

Core AI concepts

- CSP
- Genetic Algorithm
- Agents & Environments

Neural-network add-on

Predict team imbalance or project-risk level using features such as:

- skill distribution,
- availability overlap,
- number of beginners vs experienced members,
- task difficulty mix.

Inputs

- member profiles,
- skills list,
- availability data,
- role preferences,
- project task list.

Outputs

- proposed teams,
- task assignments,
- team-balance score,
- warnings about weak or overloaded teams.

Suggested tools

- Python
- pandas
- python-constraint or custom solver
- scikit-learn / Keras

Minimum scope

- 15 to 25 students
- 3 to 5 teams
- at least 3 skills or role types
- one team-quality prediction model

Stretch ideas

- conflict handling,
- team preference survey,
- compare random grouping vs CSP vs GA.

Important instruction

Clearly define how a “good team” is measured. If Genetic Algorithm is used, explain the chromosome representation and fitness function.

16) Security Project: Phishing Email Detector

Tagline: Detect suspicious emails and explain the warning signs.

Scenario

Users often receive emails that try to trick them into sharing passwords or personal data. This project builds a phishing detector that classifies suspicious messages and explains why they may be risky.

What the system should do

The user enters:

- email subject,
- email body,

- optional sender/domain information.

The system should:

- classify the email as suspicious or likely safe,
- show warning indicators,
- explain the decision,
- and optionally rank the level of risk.

Core AI concepts

- Agents & Environments
- Rule-based decision flow
- Search / feature-based filtering

Neural-network add-on

A small text classifier for phishing detection using features such as:

- urgent language,
- suspicious keywords,
- message length,
- links or attachment indicators,
- sender/domain patterns.

Inputs

- email text samples,
- labels for phishing vs safe,
- optional metadata such as sender or domain.

Outputs

- phishing/safe classification,
- risk score,
- warning explanation,
- flagged suspicious phrases.

Suggested tools

- Python
- pandas
- scikit-learn / Keras
- TF-IDF or bag-of-words models
- Streamlit optional

Minimum scope

- one small labeled email dataset
- binary classification
- rule-based baseline + learned model
- explanation of the top warning factors

Stretch ideas

- multi-level severity score,
- URL-risk indicator,
- Arabic/English message support.

Important instruction

Keep the project strictly **defensive**. Focus on detection and awareness only.

Common weekly plan for all teams

Week 1: Design and proposal

Teams must complete:

- problem definition,
- user story,
- AI concepts to be used,
- input/output definition,
- dataset or simulation plan,
- team roles.

Week 2: Baseline implementation

Teams must complete:

- environment or data setup,
- first working search/CSP/game logic,
- at least one successful test case.

Week 3: AI enhancement

Teams must complete:

- improved logic or heuristics,
- neural-network component,
- comparison against baseline.

Week 4: Final integration

Teams must complete:

- final working prototype,
- evaluation and testing,
- presentation and report.

Required final deliverables

Each team must submit:

1. Source code
2. Report (4–6 pages recommended)
3. Presentation slides
4. Live demo
5. Readme with run instructions

Suggested grading rubric

Criterion	Marks
Problem definition and realism	10
Correct use of AI concepts	25
Implementation quality	20
Neural-network component	10
Testing and evaluation	15
Demo and presentation	10
Report and documentation	10
Total	100

Important advice to students

A smaller project done well is better than a large project with weak implementation.
Your goal is not to build a commercial product. Your goal is to demonstrate good AI problem formulation, correct implementation, and clear evaluation.